Pre-Si Design Verification

SystemVerilog Constrained-Random Interview Questions

38 questions with complete SystemVerilog answers

Q1 Given 100 tokens with unique values between 1–1000, write an SV class to randomly pick 5 to 10 tokens.

Track used tokens across randomization calls; enforce uniqueness and size constraints.

Answer:

```
class A;
  int unsigned range_0, range_1;
  rand int unsigned number[], size;
  int unsigned used[$];
  function new(int start_address, int end_address);
    range_0 = start_address;
    range_1 = end_address;
  endfunction
  constraint c {
    size inside {5, 10};
    number.size == size;
    foreach (number[i]) {
      number[i] <= range_1;
      number[i] >= range_0;
    }
    size <= 100 - used.size;
  }
  function void post_randomize();
    foreach (number[i])
      used.push_back(number[i]);
  endfunction
endclass
module top;
  A h = new(0, 1000);
  bit error = 0;
  initial
    while (!error) begin
      error = !(h.randomize());
      $display("%p", h);
    end
endmodule
```

Q2 Generate an array with 10 elements (values 1–100) such that exactly three of them are the same and the rest are unique.

Answer:

```
class my_rand;
  rand int unsigned array[10];
  constraint C1 {
    foreach (array[i]) {
      array[i] inside {[1:100]};
```

```
      if (i > 0 && i < 3)
        array[i] == array[i-1];
      else {
        foreach (array[j])
          if (i != j && j > 3) array[i] != array[j];
      }
    }
  }
  function void post_randomize();
    array.shuffle();
  endfunction
endclass
module Tb;
  my_rand M1;
  initial begin
    M1 = new();
    if (M1.randomize())
      foreach (M1.array[i])
        $display("%d", M1.array[i]);
  end
endmodule
```

Q3 Generate values from different ranges with different distributions without using the 'dist' keyword.

Use a helper rand variable to act as a probability bucket.

Answer:

```
class random_dist #(DIST1=5, DIST2=10, DIST3=20);
  rand int my_number;
  rand bit [5:0] dist_factor;
  constraint custom_dist {
    dist_factor inside {[0:DIST1+DIST2+DIST3]};
    if (dist_factor <= DIST1)
      my_number inside {[10:200]};
    else if (dist_factor > DIST1 && dist_factor < DIST1+DIST2)
      my_number inside {[200:350]};
    else
      my_number inside {[400:1000]};
  }
endclass
module my_testbench;
  random_dist #(10,20,30) A1;
  initial begin
    A1 = new();
    repeat(10)
      if (A1.randomize())
        $display("A1.my_number = %0d", A1.my_number);
```

```
  end
endmodule
```

Q4 Create an array of 10 elements whose sum is 100, without using the sum() method.

Distribute a credit budget across elements; the last element absorbs remaining credits.

Answer:

```
class sum;
  rand int array[];
  rand int unsigned ArraySize;
  rand int credits;
  constraint ArraySize_c {
    soft ArraySize == 5;
    array.size() == ArraySize;
  }
  constraint total_sum_c { soft credits == 50; }
  function int array_val(int unsigned index);
    int ArrayItem;
    if (index < (ArraySize - 1)) begin
      std::randomize(ArrayItem) with { ArrayItem inside {[0:credits]}; };
      credits = credits - ArrayItem;
    end else
      ArrayItem = credits;
    return ArrayItem;
  endfunction
  function void post_randomize();
    foreach (array[idx]) begin
      array[idx] = array_val(idx);
      $display("array[%0d] = %0d", idx, array[idx]);
    end
    credits = 100;
  endfunction
endclass
module tb;
  initial begin
    sum sum1 = new();
    repeat(2) begin
      sum1.randomize() with { ArraySize == 10; credits == 100; };
      $display("Sum of Array = %0d", sum1.array.sum());
    end
  end
endmodule
```

Q5 Write a constraint random class to generate a 3×3 matrix with all unique values.

Answer:

```
class matrix_3d;
  rand int array[3][3];
  rand int my_arr[3][3];

  constraint C1 {
    foreach (array[i,j]) {
      array[i][j] inside {[1:9]};
      my_arr[i][j] inside {[1:9]};
      foreach (array[ ,j1])
        if (j != j1) array[i][j] != array[i][j1];
      foreach (array[i1,])
        if (i != i1) array[i][j] != array[i1][j];
      foreach (array[i1,j1])
        if (i1 != i && j1 != j) array[i1][j1] != array[i][j];
    }
    unique {my_arr};
  }
endclass

module Tb;
  initial begin
    matrix_3d A1 = new();
    A1.randomize();
    $display("array = %p", A1.array);
    $display("my_arr = %p", A1.my_arr);
  end
endmodule
```

Q6 Write a constraint to generate a 3×3×3 matrix with all unique elements.

Answer:

```
class matrix_3d;
  rand int array[3][3][3];
  rand int my_arr[3][3][3];

  constraint C1 {
    foreach (array[i,j,k]) {
      array[i][j][k] inside {[1:27]};
      my_arr[i][j][k] inside {[1:27]};
      foreach (array[ , ,k1]) if (k != k1) array[i][j][k] != array[i][j][k1];
      foreach (array[ ,j1, ]) if (j != j1) array[i][j][k] != array[i][j1][k];
      foreach (array[i1, , ]) if (i != i1) array[i][j][k] != array[i1][j][k];
      foreach (array[i1,j1, ]) if (i1!=i && j1!=j) array[i1][j1][k] != array[i][j][k];
      foreach (array[i1, ,k1]) if (i1!=i && k1!=k) array[i1][j][k1] != array[i][j][k];
      foreach (array[ ,j1,k1]) if (j1!=j && k1!=k) array[i][j1][k1] != array[i][j][k];
      foreach (array[i1,j1,k1])
        if (i1!=i && j1!=j && k1!=k) array[i1][j1][k1] != array[i][j][k];
    }
    unique {my_arr};
  }
```

```
endclass
module Tb;
  initial begin
    matrix_3d A1 = new();
    A1.randomize();
    $display("array = %p", A1.array);
    $display("my_arr = %p", A1.my_arr);
  end
endmodule
```

Q7 **Write constraints to generate an M×N matrix whose elements are 0 or 1 and whose total sum is less than MAX_SUM.**

Answer:

```
class matrix_3d #(M=10, N=10, MAX_SUM=10);
  rand int array[][];
  constraint size_1 {
    array.size == M;
    foreach (array[i]) array[i].size == N;
  }
  constraint C2 {
    foreach (array[i,j]) array[i][j] inside {0, 1};
  }
  constraint sum_C3 {
    array.sum() with (item.sum()) == MAX_SUM;
  }
endclass
module Tb;
  initial begin
    matrix_3d A1 = new();
    A1.randomize();
    $display("A1 = %p", A1.array);
  end
endmodule
```

Q8 **Write a constraint to solve the 8-Queens problem (place 8 queens on an 8×8 chessboard so no queen threatens another).**

Answer:

```
class Queen #(int N=8);
  rand int Q[N];
  constraint Q_valid_range {
    foreach (Q[i]) Q[i] inside {[1:N]};
  }
  constraint Q_unique {
```

```
    foreach (Q[i])
      foreach (Q[j])
        if (i != j) Q[i] != Q[j];
  }
  constraint Q_no_diagonal {
    foreach (Q[i])
      foreach (Q[j])
        if (i != j)
          !((Q[i]-Q[j]) inside {i-j, j-i});
  }
endclass
module TB;
  initial begin
    Queen myQ = new();
    repeat(10) begin
      myQ.randomize();
      foreach (myQ.Q[i])
        $display("Coordinates: (%0d, %0d)", i, myQ.Q[i]);
    end
  end
endmodule
```

Q9 Write a constraint to solve the Knight's Tour problem on a 5×5 board.

Answer:

```
class knight #(int N=5);
  rand int Row[N*N];
  rand int Col[N*N];
  rand int helper_array[N*N];
  int A[N*N], q[$];
  constraint Row_valid {
    foreach (Row[i]) Row[i] inside {[0:N-1]};
    foreach (Col[i]) Col[i] inside {[0:N-1]};
  }
  constraint helper_mapping {
    foreach (helper_array[i])
      helper_array[i] == Row[i] + N*Col[i];
  }
  constraint knight_move {
    foreach (Row[i])
      if (i > 0)
        (((Row[i]-Row[i-1] > 0) ? (Row[i]-Row[i-1]) : (Row[i-1]-Row[i]))
        +((Col[i]-Col[i-1] > 0) ? (Col[i]-Col[i-1]) : (Col[i-1]-Col[i]))) == 3
        && (Row[i] != Row[i-1]) && (Col[i] != Col[i-1]);
  }
  function void post_randomize();
    for (int i = 0; i < N**2; i++) begin
```

```
      q = helper_array.find_index with (item == i);
      A[i] = q.pop_front();
    end
  endfunction
endclass
```

Q10 Write an SV constraint to limit the sum of odd array elements to 30 and the sum of even elements to 60.

Answer:

```
class myclass #(int N=20);
  rand int array[N];
  rand int odd_queue[N];
  rand int even_queue[N];

  constraint C1 {
    foreach (array[i]) {
      array[i] inside {[0:10]};
      if (array[i] % 2) {
        odd_queue[i] == array[i];
        even_queue[i] == 0;
      } else {
        even_queue[i] == array[i];
        odd_queue[i] == 0;
      }
    }
    odd_queue.sum == 30;
    even_queue.sum == 60;
  }
endclass

module TB;
  initial begin
    myclass #(30) A = new;
    if (A.randomize())
      foreach (A.array[i])
        $display("A[%0d]= %0d", i, A.array[i]);
  end
endmodule
```

Q11 Write an SV constraint to find all solutions of a Tic-Tac-Toe game (X win, O win, or Draw).

Use +1 for X, -1 for O. Row/column/diagonal sums of ±N indicate a win.

Answer:

```
typedef enum bit [1:0] {X_Win, O_Win, Draw} result_type_t;

class c1 #(int N=3);
  rand int grid[N][N];
```

```
  rand int trans_grid[N][N];
  rand int grid_row_sum[N];
  rand int diagonal[N], anti_diagonal[N];
  rand result_type_t result_type;
  rand bit X_favor, O_favor, Novacancy_Case;

  constraint c1 { foreach (grid[i,j]) grid[i][j] inside {[-1:1]}; }
  constraint c2 { foreach (grid[i,j]) grid[i][j] == trans_grid[j][i]; }
  constraint c3 { result_type dist {X_Win:=4, O_Win:=4, Draw:=2}; }
  constraint c4 { foreach (grid_row_sum[i]) grid_row_sum[i] == grid[i].sum(); }
  constraint c5 { grid_row_sum.sum() inside {[-1:1]}; }
  constraint c6 { foreach (diagonal[i]) diagonal[i] == grid[i][i]; }
  constraint c7 { foreach (anti_diagonal[i]) anti_diagonal[i] == grid[i][N-1-i]; }

  constraint c8 {
    X_favor <-> (grid[0].sum()==N || grid[1].sum()==N || grid[2].sum()==N
                 || trans_grid[0].sum()==N || trans_grid[1].sum()==N
                 || trans_grid[2].sum()==N
                 || diagonal.sum()==N || anti_diagonal.sum()==N);
    O_favor <-> (grid[0].sum()==-N || grid[1].sum()==-N || grid[2].sum()==-N
                 || trans_grid[0].sum()==-N || trans_grid[1].sum()==-N
                 || trans_grid[2].sum()==-N
                 || diagonal.sum()==-N || anti_diagonal.sum()==-N);
    foreach (grid[i,j]) Novacancy_Case <-> grid[i][j] != 0;
  }

  constraint c9 {
    solve result_type before X_favor;
    solve result_type before O_favor;
    solve result_type before Novacancy_Case;
    (result_type == Draw) -> !X_favor && !O_favor && Novacancy_Case;
    (result_type == X_Win) -> X_favor && !O_favor;
    (result_type == O_Win) -> O_favor && !X_favor;
  }
endclass
```

Q12 Write an SV constraint to generate phone numbers (Operator, Emergency 911, Local, International).

Answer:

```
typedef enum {Operator, Emergency, Local, Intl} num_type;

class myPhoneBook;
  rand bit [34:0] val;
  rand num_type Type;

  constraint C1 {
    (Type == Operator) -> val == 0;
    (Type == Emergency) -> val == 911;
    (Type == Local) -> val inside {[1:910], [912:9999999999]};
    (Type == Intl) -> val inside {[35'd1_0000_000_000:35'd1_9999_999_999]};
  }
```

```
  constraint C2 {
    Type dist {Operator:=1, Emergency:=1, Local:=4, Intl:=4};
  }
endclass
module TB;
  initial begin
    myPhoneBook PB = new();
    repeat(20) begin
      PB.randomize();
      $display("%s Phone Num: %0d", PB.Type.name(), PB.val);
    end
  end
endmodule
```

Q13 Write an SV program to create a linked list of 20 nodes.

Answer:

```
program linked_list_prog;
  class myList;
    int data;
    myList next_item;
    function void set_data(int d); this.data = d; endfunction
    function void link_item(myList n); this.next_item = n; endfunction
    function void print_data();
      $display("data: %0d -> next: %0d", this.data, next_item.data);
    endfunction
  endclass
  initial begin
    myList List[20];
    foreach (List[i]) begin
      List[i] = new();
      List[i].set_data(i + 1);
      if (i > 0) List[i-1].link_item(List[i]);
    end
    foreach (List[i])
      if (i > 0) List[i-1].print_data();
  end
endprogram
```

Q14 Write a constraint for a 9×9 array composed of nine 3×3 sub-arrays, where the maximum value of each sub-array is unique.

Answer:

```
class my_class;
  rand int unsigned Arr[9][3][3];
```

```
  rand int max_val[9];
  int Final_Array[9][9];
  constraint uniq_max {
    foreach (max_val[i]) max_val[i] inside {[1:100]};
    unique {max_val};
    unique {Arr};
  }
  constraint sub_matrix {
    foreach (Arr[i,j,k]) Arr[i][j][k] < max_val[i];
  }
  function void post_randomize();
    for (int i=0; i<3; i++)
      for (int j=0; j<3; j++)
        Final_Array[3*i+j] = {Arr[3*i][j], Arr[3*i+1][j], Arr[3*i+2][j]};
  endfunction
endclass
```

Q15 **Write a constraint to generate 5 even and 5 odd numbers alternately in a 100-element array (repeating pattern of 10).**

Answer:

```
class rand_numbers;
  rand int Arr[100];
  rand bit k; // determines which half starts odd
  constraint C1 {
    foreach (Arr[i]) Arr[i] inside {[0:100]};
  }
  constraint odd_even_alt {
    foreach (Arr[i])
      if (i % 10 < 5) (Arr[i] & 1) == k;
      else (Arr[i] & 1) == !k;
  }
endclass
module TB;
  initial begin
    rand_numbers A = new;
    if (A.randomize())
      $display("%p", A.Arr);
  end
endmodule
```

Q16 **Given two arrays of sent and received message timestamps, calculate per-message latency, max latency, and pending messages.**

Answer:

```
class rand_numbers #(parameter N=10);
  int sent_msg[N];
  int rcvd_msg[N];
  int latency[N];
  function void calculate_latencies();
    foreach (sent_msg[i])
      latency[i] = rcvd_msg[i] - sent_msg[i];
  endfunction
  function int max_latency();
    max_latency = 0;
    foreach (latency[i])
      if (latency[i] > max_latency) max_latency = latency[i];
  endfunction
  function int pending_msgs();
    // Count entries where rcvd_msg is 0 (not yet received)
    pending_msgs = 0;
    foreach (rcvd_msg[i])
      if (rcvd_msg[i] == 0) pending_msgs++;
  endfunction
endclass
```

Q17 **Model colors with typedef enum and constrain so that the last 3 drawn colors (YELLOW, ORANGE, RED) do not repeat.**

Answer:

```
typedef enum {VIOLET, INDIGO, BLUE, GREEN, YELLOW, ORANGE, RED} color_t;
class Color;
  rand color_t myColor;
  color_t Used[$];
  constraint non_repeat { unique {myColor, Used}; }
  function void post_randomize();
    if (myColor == YELLOW || myColor == ORANGE || myColor == RED)
      Used.push_back(myColor);
  endfunction
endclass
program my_tb;
  initial begin
    Color C = new;
    repeat(100)
      if (C.randomize())
        $display("%p", C.myColor);
  end
endprogram
```

Q18 Given a 4 KB memory, write an SV class to randomly allocate it into 4 non-overlapping regions of random sizes (minimum 1 B each).

Answer:

```
class memory;
  rand bit [11:0] range[4];
  constraint C1 {
    foreach (range[i]) range[i] >= 1;
    range.sum() with (48'(item)) == 4096;
  }
  function void post_randomize();
    int Boundary[5];
    Boundary[0] = 0;
    foreach (Boundary[i])
      if (i > 0) begin
        Boundary[i] = Boundary[i-1] + range[i-1];
        $display("Region %0d: [%0d, %0d)", i-1, Boundary[i-1], Boundary[i]);
      end
  endfunction
endclass
```

Q19 Implement randc behaviour without using the randc keyword (use 'unique').

Answer:

```
class Randc;
  rand bit [3:0] myVar;
  bit [3:0] temp[$];
  constraint norepeat { unique {temp, myVar}; }
  function void post_randomize();
    temp.push_back(myVar);
    if (temp.size() >= 16) temp = {};
  endfunction
endclass
program my_prog;
  initial begin
    Randc C = new();
    repeat(25) begin
      C.randomize();
      $display("myVar = %0h", C.myVar);
    end
  end
endprogram
```

Q20 Implement randc behaviour without using either randc or the 'unique' keyword.

Answer:

```
class Randc;
  rand bit [3:0] myVar;
  bit [3:0] temp[$];
  constraint norepeat {
    foreach (temp[i]) myVar != temp[i];
  }
  function void post_randomize();
    temp.push_back(myVar);
    if (temp.size() >= 16) temp = {};
  endfunction
endclass
```

Q21 Create a uvm_seq_item to generate a 32-bit variable containing exactly 7 consecutive 1s (anywhere in the word, wrapping allowed).

Use a 38-bit shift register to handle wrap-around, then fold into 32 bits.

Answer:

```
class uvm_seq_item;
  rand bit [37:0] myReg;
  bit [31:0] final_reg;
  rand int unsigned index;
  constraint seven_ones {
    solve index before myReg;
    myReg == (7'b1111111 <<< index);
    index <= 31;
  }
  function void post_randomize();
    for (int i = 0; i < 38; i++)
      final_reg[i % 32] = myReg[i];
  endfunction
endclass
program my_prog;
  initial begin
    uvm_seq_item Txn = new;
    repeat(10) begin
      Txn.randomize();
      $display("FinalReg: %b", Txn.final_reg);
    end
  end
endprogram
```

Q22 Write an SV program to print the longest contiguous sequence of each character in a string.

Answer:

```
class print_seq;
  string Str;
  int longest_sequences[byte];
  byte prev_char = 0;
  int curr_seq_len = 0;
  function void print_sequence(string S);
    for (int i = 0; i < S.len(); i++) begin
      byte curr = S[i];
      if (curr == prev_char)
        curr_seq_len++;
      else begin
        if (prev_char != 0) begin
          if (!longest_sequences.exists(prev_char) ||
               curr_seq_len > longest_sequences[prev_char])
            longest_sequences[prev_char] = curr_seq_len;
        end
        curr_seq_len = 1;
      end
      prev_char = curr;
    end
    if (prev_char != 0) begin
      if (!longest_sequences.exists(prev_char) ||
           curr_seq_len > longest_sequences[prev_char])
        longest_sequences[prev_char] = curr_seq_len;
    end
    foreach (longest_sequences[i]) begin
      string tmp = "";
      for (int j = 0; j < longest_sequences[i]; j++) tmp = {tmp, string'(i)};
      $display("%s", tmp);
    end
  endfunction
endclass
```

Q23 SV constraint to solve the Rat-in-a-Maze problem: generate a solvable 5×5 maze (at least one path from [0,0] to [N-1,N-1]).

Two solutions shown: recursive solver in constraint function, and direction-vector approach.

Answer:

```
// Solution 1 – recursive solver
class Maze #(int N=5);
  rand bit Arr[N][N];
  constraint C0 { Arr[0][0] == 1; Arr[N-1][N-1] == 1; }
  constraint C1 { 1 == solveMaze(Arr, 0, 0); }
```

```
  function automatic bit solveMaze(bit A[N][N], int x, int y);
    if (x==N-1 && y==N-1 && A[x][y]) return 1;
    if (x<N && y<N && A[x][y]) begin
      if (solveMaze(A, x+1, y)) return 1;
      if (solveMaze(A, x, y+1)) return 1;
    end
    return 0;
  endfunction
endclass
// Solution 2 – direction vectors
class maze #(parameter SIZE=5);
  bit arr[SIZE-1:0][SIZE-1:0];
  rand bit dr[];
  rand int len[];
  constraint c0 { dr.size inside {[2:SIZE*2-2]}; len.size == dr.size; }
  constraint c1 { foreach (len[i]) len[i] inside {[1:SIZE-1]}; len.sum == (SIZE*2)-2; }
  constraint c2 { len.sum with (dr[item.index]==1 ? item : 0) == (SIZE*2-2)/2; }
  constraint c3 { len.sum with (dr[item.index]==0 ? item : 0) == (SIZE*2-2)/2; }
  function void post_randomize();
    int ii, jj;
    arr[0][0] = 1; arr[SIZE-1][SIZE-1] = 1;
    foreach (dr[i]) begin
      if (dr[i]) repeat(len[i]) begin arr[ii][jj]=1; ii++; end
      else repeat(len[i]) begin arr[ii][jj]=1; jj++; end
    end
  endfunction
endclass
```

Q24 Write an SV constraint to generate a valid SUDOKU puzzle.

Two implementations: explicit 3×3 sub-array mapping, and compact helper-array approach.

Answer:

```
// Compact implementation
class sudoku_cls;
  rand int unsigned sudoku[9][9];
  rand int unsigned helper[9][9];
  rand int sub_array[9][9];
  constraint C0 {
    foreach (sudoku[i]) unique {sudoku[i]};
    foreach (helper[i]) unique {helper[i]};
    foreach (helper[i,j]) {
      helper[i][j] == sudoku[j][i];
      helper[i][j] inside {[1:9]};
    }
    foreach (sub_array[i,j])
      sudoku[3*(i/3)+j/3][3*(i%3)+j%3] == sub_array[i][j];
  }
```

```
  function void post_randomize();
    $display("===== SUDOKU =====");
    foreach (sudoku[i]) $display("%p", sudoku[i]);
  endfunction
endclass
```

Q25 Randomize a stream of CPU commands where WR instructions are at least 3 cycles apart and RD instructions at least 6 cycles apart.

Answer:

```
typedef enum bit [1:0] {IDLE, WR, RD} istr;
class cpu_instr;
  rand istr instr[];
  constraint c_0 { instr.size inside {[10:25]}; }
  constraint c_1 {
    foreach (instr[i])
      foreach (instr[j]) {
        if (i!=j && instr[i]==WR && instr[j]==WR)
          (i-j > 3) || (i-j < -3);
        if (i!=j && instr[i]==RD && instr[j]==RD)
          (i-j > 6) || (i-j < -6);
      }
  }
  function void post_randomize();
    $display("instr: %p", instr);
  endfunction
endclass
```

Q26 Write a constraint for a 2-D matrix where adjacent elements (horizontal, vertical, and diagonal) are unique.

Answer:

```
class Matrix #(SIZE=10);
  rand int arr[SIZE][SIZE];
  rand int reverse[SIZE][SIZE];
  constraint c0 {
    foreach (arr[i,j]) {
      arr[i][j] inside {[1:100]};
      arr[i][j] == reverse[j][i];
      if (j > 0) arr[i][j] != arr[i][j-1];
      if (i > 0 && j > 0) arr[i][j] != arr[i-1][j-1];
    }
    foreach (reverse[i,j])
      if (j > 0) reverse[i][j] != reverse[i][j-1];
  }
```

```
  function void post_randomize();
    foreach (arr[i]) $display("%p", arr[i]);
  endfunction
endclass
```

Q27 **Write a constraint for an N-bit variable where the number of 1s equals the number of 0s (N must be even).**

Answer:

```
class my_class #(N=16);
  rand bit [N-1:0] rand_bit;
  rand int arr[N];

  constraint C0 {
    arr.sum() with (int'(item == 1)) == arr.sum() with (int'(item == 0));
    foreach (arr[i]) arr[i] == rand_bit[i];
  }

  function void post_randomize();
    $display("%b SUM=%0d", rand_bit, arr.sum());
  endfunction
endclass
```

Q28 **Generate a 300-element array (values 0–5, each at least 40 times) with no two consecutive 0s.**

Answer:

```
class my_class #(N=300);
  rand int arr[N];

  constraint C0 {
    foreach (arr[i]) arr[i] inside {[0:5]};
    arr.sum() with (int'(item==0)) > 40;
    arr.sum() with (int'(item==1)) > 40;
    arr.sum() with (int'(item==2)) > 40;
    arr.sum() with (int'(item==3)) > 40;
    arr.sum() with (int'(item==4)) > 40;
    arr.sum() with (int'(item==5)) > 40;
    foreach (arr[i])
      if (i > 0) (arr[i]==0) -> (arr[i-1] != 0);
  }
endclass
```

Q29 **Same as above but 0 may repeat; non-zero values (1–5) must not appear consecutively.**

Answer:

```
class my_class #(N=300);
  rand int arr[N];
  constraint C0 {
    foreach (arr[i]) arr[i] inside {[0:5]};
    arr.sum() with (int'(item==0)) > 40;
    arr.sum() with (int'(item==1)) > 40;
    arr.sum() with (int'(item==2)) > 40;
    arr.sum() with (int'(item==3)) > 40;
    arr.sum() with (int'(item==4)) > 40;
    arr.sum() with (int'(item==5)) > 40;
    foreach (arr[i])
      if (i > 0) (arr[i] != 0) -> (arr[i-1] != arr[i]);
  }
endclass
```

Q30 Pick a ball from 10 colored balls such that the same color is not repeated in the next 3 draws.

Answer:

```
typedef enum bit [3:0] {
  VIOLET, INDIGO, BLUE, GREEN, YELLOW,
  ORANGE, RED, PINK, BLACK, WHITE
} color;
class my_class;
  rand color Color;
  color myQ[$];
  constraint C0 { unique {Color, myQ}; }
  function void post_randomize();
    $display("Color Picked: %p", Color);
    if (myQ.size() > 2) myQ.pop_front();
    myQ.push_back(Color);
  endfunction
endclass
```

Q31 Divide the elements of one queue into three queues such that each sub-queue contains unique elements.

Answer:

```
class my_class;
  rand int A[$];
  rand int a[$], b[$], c[$];
  constraint C0 {
    A.size() inside {[10:30]};
    a.size() inside {[0:30]};
    b.size() inside {[0:30]};
    c.size() inside {[0:30]};
```

```
    A.size() == a.size() + b.size() + c.size();
    unique {a}; unique {b}; unique {c};
    foreach (A[i]) {
      A[i] inside {[1:30]};
      if (i < a.size())
        A[i] == a[i];
      else if (i < a.size() + b.size())
        A[i] == b[i - a.size()];
      else
        A[i] == c[i - a.size() - b.size()];
    }
  }
  function void post_randomize();
    $display("a=%p b=%p c=%p", a, b, c);
  endfunction
endclass
```

Q32 **Generate an 8-bit variable: if the previous value was odd, the next value is even with 75% probability; otherwise even with 25% probability.**

Answer:

```
class myVar;
  rand bit [7:0] V;
  int past_var = 0;
  constraint C0 {
    if (past_var % 2)
      V[0] dist {0:=75, 1:=25};
    else
      V[0] dist {0:=25, 1:=75};
  }
  function void post_randomize();
    past_var = int'(V);
    $display("My Variable: %0d", V);
  endfunction
endclass
```

Q33 **Generate a stream of random numbers where the most recent 4 numbers are always unique.**

Answer:

```
class myVar;
  rand int Rand_num;
  int past_4[$];
  constraint C0 {
    Rand_num inside {[1:7]};
    unique {past_4, Rand_num};
```

```
  }
  function void post_randomize();
    if (past_4.size() > 3) past_4.pop_front();
    past_4.push_back(Rand_num);
    $display("Number: %0d", Rand_num);
  endfunction
endclass
```

Q34 Generate two dynamic arrays: array1 size [6:9] in ascending order; array2 same size with values shuffled from array1.

Answer:

```
class myVar;
  rand int array1[], array2[];
  constraint C0 {
    array1.size inside {[6:9]};
    array2.size == array1.size;
  }
  constraint C1 {
    foreach (array1[i]) {
      array1[i] inside {[1:1000]};
      foreach (array1[j])
        if (i > j) array1[i] > array1[j]; // ascending
      array2[i] == array1[i];
    }
  }
  function void post_randomize();
    array2.shuffle();
    $display("Array1: %p", array1);
    $display("Array2: %p", array2);
  endfunction
endclass
```

Q35 Constrain a 4-bit variable so that the lower two bits being identical has only a 5% probability.

Lower two bits equal when both are 00 or both are 11.

Answer:

```
class myVar;
  rand bit [3:0] Var;
  static int counter;
  constraint C0 {
    Var[1:0] dist {2'b00:=5, 2'b11:=5, [2'b01:2'b10]:/190};
  }
  function void post_randomize();
```

```
    if (Var[1:0] == 2'b11 || Var[1:0] == 2'b00) counter++;
    $display("MyVar: %b (same-bits count: %0d)", Var, counter);
  endfunction
endclass
```

Q36 Constrain a 10-bit variable 'en' so that each popcount (1 through 10) occurs with exactly 10% probability.

Answer:

```
class myVar;
  rand bit [9:0] en;
  rand int unsigned dist_var;
  static int array[int];
  constraint C0 {
    solve dist_var before en;
    dist_var inside {[1:10]};
    $countones(en) == dist_var;
  }
  function void post_randomize();
    if (array.exists($countones(en)))
      array[$countones(en)]++;
    else
      array[$countones(en)] = 1;
  endfunction
endclass
program my_prog;
  myVar Var;
  initial begin
    Var = new;
    repeat(10000) Var.randomize();
    foreach (Var.array[i])
      $display("popcount %0d: %0d times", i, Var.array[i]);
  end
endprogram
```

Q37 Write a constraint to generate a number whose binary representation has all 1s grouped together (contiguous block of 1s).

Answer:

```
class myVar #(N=50);
  rand bit [N-1:0] Var;
  rand int unsigned no_of_ones;
  rand int unsigned pos;
  constraint C0 {
    solve no_of_ones before Var;
    solve pos before Var;
```

```
    no_of_ones inside {[1:N]};
    pos inside {[0:N-1]};
    pos + no_of_ones <= N - 1;
    foreach (Var[i]) {
      if (i < pos || i >= pos + no_of_ones) Var[i] == 0;
      else Var[i] == 1;
    }
  }
  function void post_randomize();
    $display("Number: %b (ones: %0d)", Var, $countones(Var));
  endfunction
endclass
```

Q38 **Write a constraint so that every even-indexed element of an array is even, and every odd-indexed element is odd.**

Answer:

```
class myVar #(N=20);
  rand int arr[N];
  constraint C0 {
    foreach (arr[i]) {
      arr[i] inside {[0:1000]};
      (i % 2) != (arr[i] % 2); // index parity != value parity
    }
  }
  function void post_randomize();
    $display("Array: %p", arr);
  endfunction
endclass
```